

Tarea 1 2026

Count-Min Sketch y CountSketch en ventana deslizante

Lucas Morales Oyanedel

1 de octubre de 2026

Resumen

Se implementan Count-Min Sketch (CMS) y CountSketch (CS) sobre una ventana deslizante construida como anillo de subventanas, aprovechando la linealidad de ambos sketches. Se evalúan sobre la traza MAWI samplepoint-F (2018-12-03 14:00 JST) con dos ataques sintéticos inyectados (DDoS y Scan, semilla 42). Se estudia la estimación de frecuencia, la detección de heavy hitters y la estimación de la variación de frecuencia Δf , comparando el error de cada sketch al variar el ancho w .

1. Introducción

El objetivo es monitorear tráfico de red en una ventana de $W = 60$ s que avanza en pasos de $\delta = 10$ s, usando memoria sublineal. La ventana se divide en $m = W/\delta$ subventanas, cada una con su propio sketch, y se mantiene un sketch acumulado A_j . La traza base es <https://mawi.wide.ad.jp/mawi/samplepoint-F/2018/201812031400.pcap.gz>; sobre ella se inyectan un ataque DDoS y un ataque Scan.

2. Estructura del proyecto

Archivo	Origen	Descripción
<code>pcap2bin.cpp</code>	entregado	Convierte <code>pcap</code> a registros de 24 B
<code>exact_hh.cpp</code>	entregado	Ground truth exacto por ventana
<code>inject_attack.py</code>	entregado	Inyecta ataques DDoS y Scan
<code>sketches.hpp</code>	nuevo	Implementación de CMS y CS
<code>tarea1.cpp</code>	nuevo	Driver con ventana deslizante y anillo
<code>run_experiments.py</code>	nuevo	Pipeline de experimentos y gráficos
<code>Makefile</code>	nuevo	Reglas <code>all</code> , <code>run</code> , <code>run-plots</code> , <code>clean</code>

Cuadro 1: Archivos del proyecto.

3. Metodología

3.1. Datos y ataques

Se convierte la traza a binario (`pcap2bin`) y se inyectan dos ataques con `inject_attack.py`, ambos desde $t = 300$ s durante 30 s:

- **DDoS:** 10 000 pps desde 4 000 fuentes hacia una víctima. Clave: `dst`.
- **Scan:** 8 000 pps desde un atacante hacia 60 000 destinos. Clave: `src`.

3.2. Ventana deslizante y linealidad

Ambos sketches son lineales, por lo que la ventana se mantiene como

$$A_{j+1} = A_j - S_{j-m+1} + S_{j+1}. \quad (1)$$

Cada rotación actualiza solo dos subventanas: $2dw$ operaciones, es decir $O(dw)$, independiente del número de paquetes en la ventana.

3.3. Decisiones de diseño

Clave única de 128 bits. Todas las claves (`src`, `dst`, `src24`, `src16`, `5tuple`) se representan como `unsigned __int128`, con una sola tabla hash y un solo tipo de sketch. La 5-tupla usa 104 bits ($32+32+16+16+8$).

Alineación de subventanas. $q(ts) = \lceil (ts - t_0)/p \rceil$ con $q \geq 1$. Un paquete en el borde $ts = t_0 + kp$ pertenece a la subventana que termina en ese borde. Usar $\lfloor dt/p \rfloor + 1$ desalinea el anillo y desfasa en una subventana la salida del ataque.

Rotación. En cada τ_j se cargan primero los paquetes hasta τ_j (entra S_j) y luego se expira la subventana más antigua (S_{j-m}).

Precarga. Antes de $\tau_0 = t_0 + W$ se cargan los paquetes de $(t_0, t_0 + W]$ en las m ranuras del anillo.

Hash. Un functor `KeyHash` con `SplitMix64` sobre las dos mitades del entero (necesario porque `std::unordered_map<__int128, T>` no compila de forma estándar en GCC). Los sketches usan `hash_pos(k, r, w) ∈ [0, w)` y `hash_sign(k, r) ∈ {−1, +1}`.

3.4. Estimadores

- **CMS:** $\min_j C[j][h_j(k)]$, truncado inferiormente en 0.
- **CS:** $\text{mediana}_j (s_j(k) C[j][h_j(k)])$, sin truncar, para permitir valores negativos en Δf .
- **CMS-mediana:** $\text{mediana}_j C[j][h_j(k)]$, sin signo.

3.5. Umbral de heavy hitter

$T_j = \lceil \varphi N_j \rceil$, con $T_j = 1$ si resulta 0. El volumen N_j se mantiene exactamente con un anillo escalar paralelo.

3.6. Verificación de N_j

Se valida que N_j de `tarea1` coincida con el de `exact_hh`, comparando `out/exact_N.csv` con `out/est_N.csv` sobre todas las ventanas.

4. Reproducción de los experimentos

Requisitos: C++17, Python 3, GNU Make, curl, zcat, g++, y numpy, pandas, matplotlib.

```
make all
curl -L -C - -O https://mawi.wide.ad.jp/mawi/samplepoint-F/2018/201812031400.pcap.gz
zcat 201812031400.pcap.gz | ./pcap2bin > traza.bin
make run # inyeccion, experimentos y graficos
make run-plots # solo regenerar graficos
```

Actividades individuales:

```
./tarea1 traza.bin --mode estimate --key src --sketch cms -w 1024 --query 198.18.0.7
--out out/est.csv
./tarea1 traza_ddos.bin --mode detect --key dst --sketch cs -w 4096 --query <
IP_victima> --out out/detect_ddos_cs_w4096.csv
./tarea1 traza_scan.bin --mode delta --key src --sketch cs -w 1024 --query <
IP_atacante> --out out/delta_scan.csv
```

Salidas: out/ (CSV por corrida), figures/ (cuatro PNG) y results/summary.csv (MRE, latencia y memoria por (ataque, sketch, w)).

5. Análisis de gráficos

5.1. Frecuencia absoluta (freq_ddos.png, freq_scan.png)

Se ejecuta `./tarea1 -mode detect` para la clave de interés en todas las subventanas τ_j , con CMS y CS en $w \in \{256, 1024, 4096\}$. Cada figura muestra 6 curvas de estimación más el valor exacto, y un panel de zoom en $t \in [300, 390]$ s. Se mide el conteo absoluto de paquetes de la clave dentro de la ventana de $W = 60$ s.

Insertar figures/freq_ddos.png (descomentar includegraphics)

Figura 1: Frecuencia exacta vs. estimada, ataque DDoS.

Interpretación:

- Una curva superpuesta al valor exacto indica alta fidelidad.
- **CMS** con w pequeño (p.ej. 256) queda sistemáticamente por encima del valor real: las colisiones solo suman y el mínimo no las compensa.
- **CS** con w pequeño fluctúa hacia arriba y hacia abajo del valor exacto (signos ± 1); al subir w a 4096 la varianza baja y converge.
- Perfil temporal: tramo bajo de tráfico de fondo, rampa ascendente de 60 s cuando el ataque entra a la ventana, meseta durante la inyección y descenso al expirar los paquetes.

5.2. Variación de frecuencia (delta_ddos.png, delta_scan.png)

Con `-mode delta` se calcula $\Delta f_j(x) = f_j(x) - f_{j-1}(x)$ para el valor exacto, CS (Δcf_j^{CS}) y CMS-mediana ($\Delta cf_j^{CMS-med}$). La figura tiene dos paneles: arriba, Δf en el tiempo; abajo, el error absoluto $|\Delta cf - \Delta f_{exact}|$ de cada método.

Insertar figures/delta_ddos.png (descomentar includegraphics)

Figura 2: Δf y error absoluto, ataque DDoS.

Interpretación:

- **Entrada:** pico positivo marcado en el instante en que el ataque empieza a impactar la ventana.
- **Salida:** al pasar $W = 60$ s desde el fin de la inyección, el tráfico expira y aparece un pico negativo simétrico.

- **Régimen estacionario:** $\Delta f \approx 0$ en la meseta del ataque y en tráfico de fondo, pues entran tantos paquetes como expiran.
- **CS vs. CMS-mediana:** CS tiene errores significativamente menores por la cancelación de ruido en diferencias; CMS-mediana, al ser heurístico, muestra picos de error más fuertes en las transiciones abruptas.

6. Preguntas obligatorias

6.1. ¿Por qué la linealidad mantiene la ventana con costo independiente de los paquetes que contiene?

La ventana es la suma de sus m subventanas. Al avanzar basta restar la que expira y sumar la nueva (ecuación 1), de modo que solo se actualizan los contadores de dos subventanas. Para d filas y ancho w son $2dw$ actualizaciones, $O(dw)$, sin recorrer de nuevo los paquetes de la ventana.

6.2. ¿Qué diferencias hay entre el error de CMS y CS al reducir w ?

Al reducir w aumentan las colisiones. En CMS solo pueden inflar la estimación, por lo que el error es mayor sobreestimación. En CS los signos hacen que las contribuciones de colisiones se cancelen en esperanza: el estimador sigue insesgado, pero crece su varianza, con errores de mayor magnitud y variaciones no monótonas.

6.3. ¿Qué ataque se detecta con mayor claridad y por qué la clave difiere?

La claridad depende de cómo la clave agrupa el tráfico. El DDoS concentra muchas fuentes hacia una víctima, así que `dst` las agrupa bajo una misma dirección. El Scan es un emisor hacia múltiples destinos, por lo que `src` agrupa su tráfico sin importar los destinos. Cada ataque se analiza con la clave que concentra mejor su comportamiento: `dst` para DDoS y `src` para Scan.

6.4. ¿Qué información adicional entrega $\Delta f_j(x)$ respecto de $f_j(x)$?

$f_j(x)$ dice cuánto tráfico de x hay en la ventana actual, pero no cómo cambió. $\Delta f_j(x)$ es positivo si aumenta y negativo si disminuye, lo que deja ver con claridad la entrada y la salida de un ataque como aumentos y disminuciones abruptas.

6.5. Comparación de $\Delta_c f_j^{CS}(x)$ y $\Delta_c f_j^{CMS-med}(x)$

CS aplica su estimador habitual a la diferencia entre subventanas porque sus contadores usan signos ± 1 ; al restar, conservan la estructura necesaria para que la mediana siga siendo aplicable a Δf . En CMS el estimador usa el mínimo y supone frecuencias no negativas; en diferencias aparecen valores negativos y el mínimo deja de ser adecuado, por lo que se usa la mediana de las filas (CMS-mediana). Esa modificación no conserva las garantías estándar de CMS, que dependen del mínimo y de la no negatividad.

7. Conclusiones

La linealidad permite mantener la ventana deslizante con costo $O(dw)$ por rotación. CS es insesgado y aprovecha la cancelación de colisiones, lo que lo hace superior para estimar Δf , mientras que CMS sobreestima de forma sistemática con anchos pequeños y su variante con mediana carece de garantías formales. La elección de clave (`dst` para DDoS, `src` para Scan) es determinante para que cada ataque se vea como un heavy hitter claro, y Δf revela sus firmas de entrada y salida.